Name: G. Jaya Sai
Roll No: 24501A4415

Date: -10-2026
Branch: CSD

Experiment - 13

Aim:

To develop an end-to-end machine learning pipeline, following the Cross-Industry Standard Process for Data Mining (CRISP-DM), that predicts whether a home-loan application will be approved from the applicant's income, loan details, credit history and personal and property information, and to compare Logistic Regression, Decision Tree, K-Nearest Neighbours (KNN) and Support Vector Machine (SVM) classifiers using accuracy, macro precision, macro recall and macro F1-score. **Project Title:** Loan Approval Prediction Using Machine Learning

1. Business Understanding

1.1 Problem Statement and Relevance

A lender must decide for every application whether to approve the loan. The decision depends on several factors acting together: how much the applicant and co-applicant earn, how large and how long the loan is, whether the applicant has a credit history that meets the lender's guidelines, and personal and property details. Checking every application by hand is slow, and a data-driven model can support the first screening of applications.

This project formulates loan approval as a supervised binary classification problem. Given eleven attributes of an application, the model predicts one of two outcomes: **Y** (loan approved) or **N** (loan not approved). **1.2 Objectives**

- Load the loan dataset and inspect its structure, missing values, duplicates and class balance.
- Explore the data visually (loan status distribution, missing values, correlation between the numerical features and approval rates by category).
- Prepare the features and the label and split the data into stratified training and testing sets.
- Build four classification pipelines (Logistic Regression, Decision Tree, KNN, SVM), including imputation, encoding and scaling, and train them on the training set.
- Evaluate the models using accuracy, macro precision, macro recall and macro F1-score, and examine the selected model with a classification report and a confusion matrix.
- Save the selected model, reload it to verify it, and demonstrate a loan-prediction function. **1.3 Expected Benefits**
- A reproducible workflow that turns application details into an approve / not-approve prediction.
- A quantified comparison of four classical classifiers on the same data, with the same preprocessing.
- Insight into which applications are easy or difficult to classify (through per-class metrics and the confusion matrix) and which attributes are linked to approval.
- A saved model and a function that show how the trained model can be reused without retraining.

1.4 Scope

- **In scope:** one tabular dataset of 614 applications; eleven input features (five numerical, six categorical); two classes; Logistic Regression, Decision Tree, KNN and SVM; a stratified 80:20 hold-out evaluation; saving and reloading the selected model.
- **Out of scope:** other algorithms, hyperparameter search, cross-validation, class re-balancing, fairness auditing, financial or legal rules of lending, and any website, API or production application. The deliverable is a notebook and a saved model file only.

1.5 Problem Formulation

The task is a supervised binary classification problem. The input is a vector of eleven features, and the output is one of two labels:

Loan_Status = f(ApplicantIncome, CoapplicantIncome, LoanAmount, Loan_Amount_Term, Credit_History, Gender, Married, Dependents, Education, Self_Employed, Property_Area) The six CRISP-DM phases map onto this project as follows.

Table 1.1: CRISP-DM phases and their realisation in this project

CRISP-DM phase	Activity in this project	Report section

Business Understanding	Define loan approval as a two-class classification task; set objectives and scope.	1
Data Understanding	Inspect the dataset, check missing values, duplicates and class balance, plot the status distribution, missing values, feature correlations and approval rates.	2
Data Preparation	Remove duplicates, drop the identifier, select features and label, rely on in-pipeline imputation, encoding and scaling, split in a stratified way.	3
Modelling	Build and train four classification pipelines.	4
Evaluation	Compute accuracy and macro precision, recall and F1; compare models; analyse the selected model by class.	5
Deployment	Save the selected model with joblib, reload it and demonstrate a prediction function.	6

2. Data Understanding

2.1 Dataset Overview

The dataset is a table of 614 records and 13 columns. Each record describes one loan application through an identifier (Loan_ID), eleven descriptive attributes and a label (Loan_Status) stating whether the loan was approved. Of the 614 applications, 422 (68.7%) were approved (Y) and 192 (31.3%) were not approved (N), so the classes are imbalanced.

2.2 Dataset Source

The notebook reads the data from the CSV file train_u6lujuX_CVtuZ9i.csv (a local copy named loan_data.csv is used in the code below). The file name and the columns are consistent with the publicly available Loan Prediction practice dataset published by Analytics Vidhya and widely re-shared on Kaggle (see References), but the file itself does not state its origin. The student should confirm the source of the file before submission.

2.3 Dataset Structure

The file was read with pd.read_csv. The first five rows are shown below.

Code

```
# Cell 2: Load dataset df =
pd.read_csv("loan_data.csv")
print("Dataset loaded
successfully")
```

Output: Dataset loaded successfully

Table 2.1: First rows of the dataset (NaN marks a missing value)

Loan_ID	Gender	Married	Dependents	Education	Self_ Employed	Applicant Income	Coapplicant Income	Loan Amount	Loan_Amt_ Term	Credit_ History	Property_ Area	Loan_ Status
LP001002	Male	No	0	Graduate	No	5849	0.0	NaN	360.0	1.0	Urban	Y
LP001003	Male	Yes	1	Graduate	No	4583	1508.0	128.0	360.0	1.0	Rural	N
LP001005	Male	Yes	0	Graduate	Yes	3000	0.0	66.0	360.0	1.0	Urban	Y
LP001006	Male	Yes	0	Not Graduate	No	2583	2358.0	120.0	360.0	1.0	Urban	Y
LP001008	Male	No	0	Graduate	No	6000	0.0	141.0	360.0	1.0	Urban	Y

The thirteen columns are Loan_ID, Gender, Married, Dependents, Education, Self_Employed, ApplicantIncome, CoapplicantIncome, LoanAmount, Loan_Amount_Term, Credit_History, Property_Area and Loan_Status. The first application has no LoanAmount value, which already shows that the data contain gaps.

2.4 Data Dictionary

The descriptions follow the public documentation of the dataset; the file itself does not define units.

Table 2.2: Data dictionary

Variable	Description	Type	Role
Loan_ID	Unique identifier of the application.	Text	Dropped (identifier)

Gender	Male / Female.	Categorical	Input feature
Married	Applicant married (Yes / No).	Categorical	Input feature
Dependents	Number of dependents (0, 1, 2, 3+).	Categorical	Input feature
Education	Graduate / Not Graduate.	Categorical	Input feature
Self_Employed	Applicant self-employed (Yes / No).	Categorical	Input feature
ApplicantIncome	Applicant's income.	Numerical	Input feature
CoapplicantIncome	Co-applicant's income (0 when there is none).	Numerical	Input feature
LoanAmount	Loan amount requested (in thousands in the public documentation).	Numerical	Input feature
Loan_Amount_Term	Loan term in months.	Numerical	Input feature
Credit_History	1 = credit history meets guidelines, 0 = it does not.	Numerical (0/1)	Input feature
Property_Area	Urban / Semiurban / Rural.	Categorical	Input feature
Loan_Status	Y = loan approved, N = not approved.	Categorical	Target (class label)

2.5 Data Quality Observations

The inspection step of the notebook checks the column names, missing values, duplicate rows, descriptive statistics and the number of records per class.

Code

```
# Cell 3: Inspect data print("Columns:",
df.columns.tolist()) print("\nMissing values:")
miss = df.isna().sum() print(miss.to_string())
print("\nDuplicate rows:",
df.duplicated().sum()) desc = df.describe().T
print("\nDescriptive statistics:")
print(desc.to_string()) print("\nLoan status
counts:")
print(df["Loan_Status"].value_counts().to_string())
```

Output (condensed in the tables below): the columns are those listed in Section 2.3; there are no duplicate rows (0); 149 cells are missing in total; the descriptive statistics are in Tables 2.3 and 2.4.

Table 2.3: Descriptive statistics of the numerical features (notebook output)

Feature	count	mean	std	min	25%	50%	75%	max
ApplicantIncome	614	5403.46	6109.04	150.00	2877.50	3812.50	5795.00	81000.00
CoapplicantIncome	614	1621.25	2926.25	0.00	0.00	1188.50	2297.25	41667.00
LoanAmount	592	146.41	85.59	9.00	100.00	128.00	168.00	700.00
Loan_Amount_Term	600	342.00	65.12	12.00	360.00	360.00	360.00	480.00
Credit_History	564	0.84	0.36	0.00	1.00	1.00	1.00	1.00

Table 2.4: Missing values per column (notebook output)

Column	Missing	% of 614	Column	Missing	% of 614
Loan_ID	0	0.0%	CoapplicantIncome	0	0.0%
Gender	13	2.1%	LoanAmount	22	3.6%
Married	3	0.5%	Loan_Amount_Term	14	2.3%

Dependents	15	2.4%	Credit_History	50	8.1%
Education	0	0.0%	Property_Area	0	0.0%
Self_Employed	32	5.2%	Loan_Status	0	0.0%
ApplicantIncome	0	0.0%			

1. **Incomplete data.** 134 of the 614 records (21.8%) have at least one missing value, 149 cells in total. Credit_History (50), Self_Employed (32) and LoanAmount (22) have the most. The label has no missing values, and there are no duplicate rows. Missing values are handled inside the model pipelines (Section 3.4).
2. **Imbalanced classes.** 68.7% of the applications are approved. A model that always answers Y would already reach about 69% accuracy, so accuracy alone is a weak yardstick here and macro-averaged metrics and per-class results are also reported.
3. **Different feature scales.** ApplicantIncome has a mean of about 5,403 and CoapplicantIncome about 1,621, while Credit_History is a 0/1 flag with mean 0.84. This matters for distance-based and margin-based models (Section 3.5).
4. **Skewed numerical features with extreme values.** ApplicantIncome has a median of 3,812.5 but a maximum of 81,000, and LoanAmount has a median of 128 but a maximum of 700. The notebook does not remove or cap outliers, so these values remain in the data.
5. **Dominant loan term.** 512 of the 600 recorded Loan_Amount_Term values are 360 months (85.3%), so this feature varies little.

2.6 Exploratory Data Analysis

Four plots were produced: status distribution, missing values, numerical correlations and approval rates by category.

Code

```
# Cell 4: Exploratory data analysis plt.figure(figsize=(6, 4)) ax =
df["Loan_Status"].value_counts().plot(kind="bar", color=["#1f77b4", "#d62728"])
plt.title("Loan Status Distribution") plt.xlabel("Loan status (Y = approved, N =
rejected)") plt.ylabel("Applications") plt.xticks(rotation=0) for p in ax.patches:
    ax.annotate(int(p.get_height()), (p.get_x() + p.get_width() / 2,
p.get_height()),                 ha="center", va="bottom") plt.tight_layout()
plt.show()

plt.figure(figsize=(9, 4)) miss[miss >
0].sort_values(ascending=False).plot(kind="bar", color="#1f77b4")
plt.title("Missing Values per Column") plt.ylabel("Missing entries")
plt.xticks(rotation=30) plt.tight_layout() plt.show()

plt.figure(figsize=(8, 6)) corr =
df.select_dtypes(include=np.number).corr()
sns.heatmap(corr, annot=True, cmap="coolwarm",
fmt=".2f") plt.title("Correlation Between Numerical
Features") plt.tight_layout() plt.show()

fig, axes = plt.subplots(1, 3, figsize=(13, 4)) for ax, c in zip(axes,
["Credit_History", "Property_Area", "Married"]):
(df.assign(y=(df["Loan_Status"] == "Y") * 100)
.groupby(c)["y"].mean().plot(kind="bar", ax=ax, color="#1f77b4"))
ax.set_title(f"Approval rate by {c}")     ax.set_ylabel("Approved (%)")
ax.set_ylim(0, 100)     ax.tick_params(axis="x", rotation=0)
plt.tight_layout() plt.show()
```

2.6.1 Loan status distribution

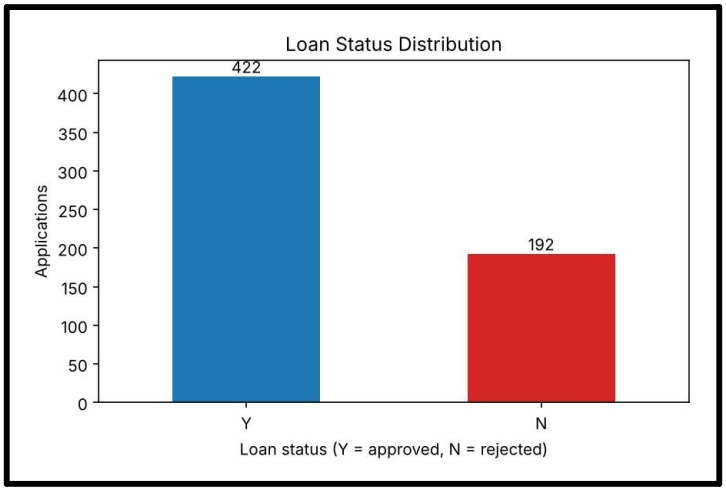


Figure 2.1: Loan status distribution (notebook output)

The bars show 422 approved (Y) and 192 not approved (N) applications, confirming the imbalance noted in Section 2.5.

2.6.2 Missing values

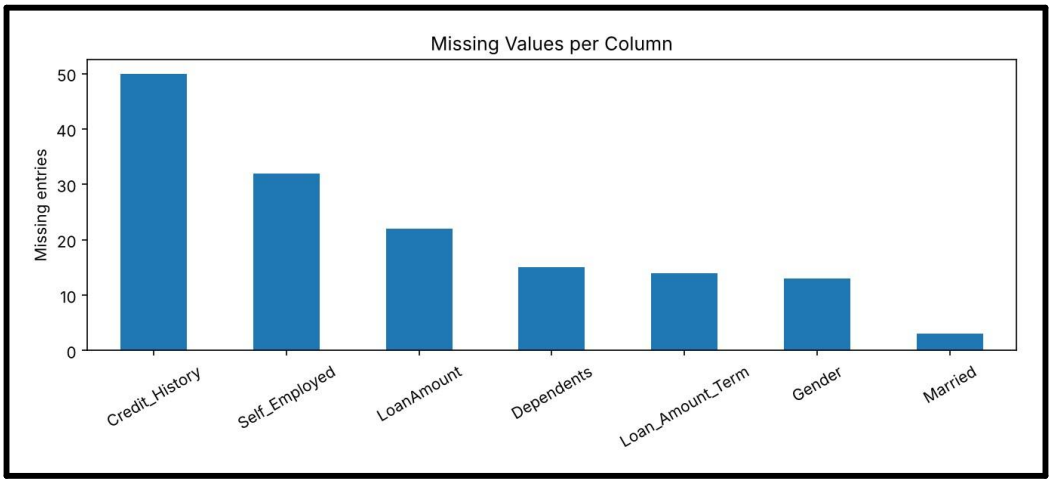


Figure 2.2: Missing values per column (notebook output)

Seven of the thirteen columns contain missing values; Credit_History has the most (50 entries, 8.1%).

2.6.3 Correlation between numerical features

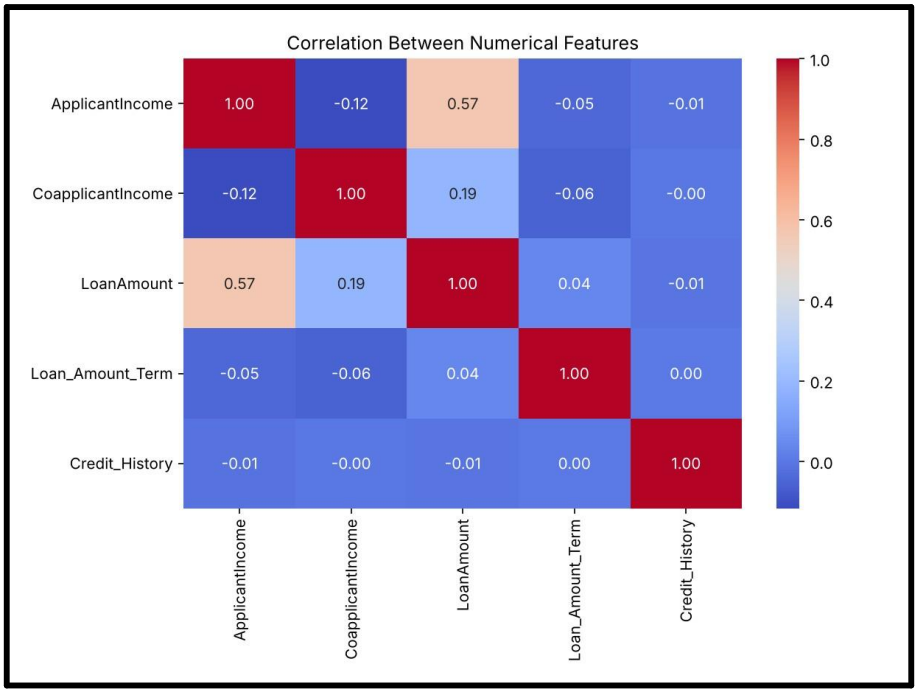


Figure 2.3: Correlation between numerical features (notebook output)

The correlation matrix covers the five numerical features only. The strongest relationships are ApplicantIncome–LoanAmount (0.57), CoapplicantIncome–LoanAmount (0.19), ApplicantIncome–CoapplicantIncome (-0.12). All other pairs have absolute correlations below about 0.2, so the numerical features are mostly weakly correlated. Because the matrix does not include the categorical features or the label, it says nothing about which features drive approval.

2.6.4 Approval rate by category

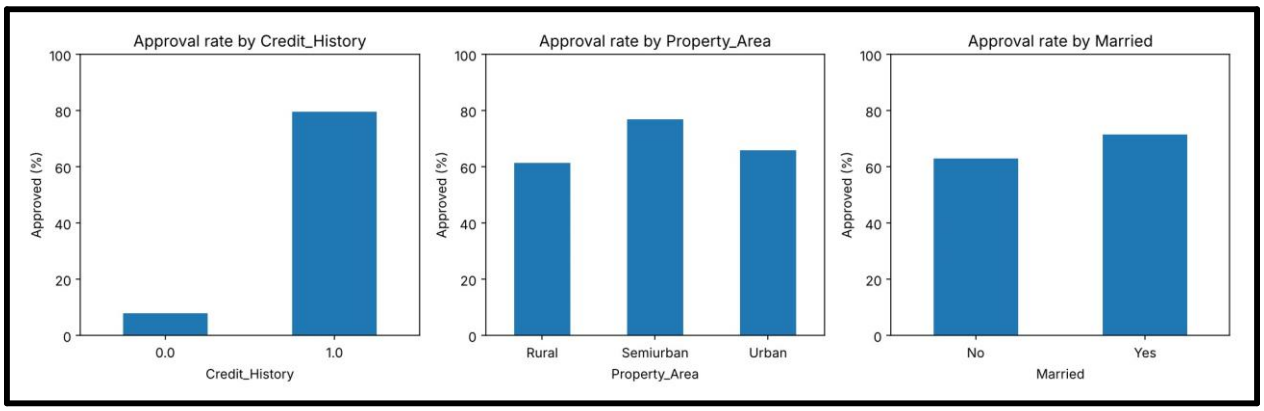


Figure 2.4: Approval rate by Credit_History, Property_Area and Married (notebook output)

Credit history stands out: 79.6% of the 475 applicants with Credit_History = 1 were approved, against only 7.9% of the 89 with Credit_History = 0. Approval is also higher in semiurban areas (76.8%) than in urban (65.8%) or rural (61.5%) ones, and higher for married applicants (71.6%) than for unmarried ones (62.9%). Graduates were approved 70.8% of the time against 61.2% for non-graduates. These are simple rates computed over records with a known value of the feature; they show association, not cause.

3. Data Preparation

3.1 Dataset Loading

The CSV file is read into a pandas DataFrame (the code and output are shown in Section 2.3).

3.2 Data Cleaning

Duplicate rows are removed and the index is reset. The notebook then checks that all eleven feature columns and the label column exist, and stops with an error if any is missing. Since the inspection found no duplicates, no row is actually removed (the shape

stays 614 records). The identifier Loan_ID is not part of the feature list and is therefore left out of the model, because a unique identifier carries no information about approval.

3.3 Feature and Target Selection

The five numerical columns and six categorical columns form the feature matrix X (the numerical columns are converted with pd.to_numeric, which would turn any non-numeric entry into NaN), and the Loan_Status column, converted to text, forms the target y.

3.4 Missing-Value Handling

Rows with an empty label are removed (none exist). Missing values in the features are filled inside each model pipeline so that the imputation is learned from the training data only: numerical columns (including the 0/1 Credit_History) receive the column median, and categorical columns receive the most frequent category. The 149 missing cells therefore remain in the data frame and are filled separately for each fitted pipeline.

Code

```
# Cell 5: Clean and prepare data
df = df.drop_duplicates().reset_index(drop=True)
numeric_features = ["ApplicantIncome", "CoapplicantIncome", "LoanAmount",
"Loan_Amount_Term", "Credit_History"] categorical_features = ["Gender", "Married",
"Dependents", "Education",                          "Self_Employed", "Property_Area"]
required_features = numeric_features + categorical_features missing_required = [c for c
in required_features + ["Loan_Status"] if c not in df.columns] if missing_required:
raise ValueError(f"Dataset is missing required columns: {missing_required}")

X = df[required_features].copy() X[numeric_features] =
X[numeric_features].apply(pd.to_numeric, errors="coerce") y =
df["Loan_Status"].astype(str)

valid = y.notna() & (y.str.strip() !=
"") X =
X.loc[valid].reset_index(drop=True) y =
y.loc[valid].reset_index(drop=True)
print("Prepared shape:", X.shape)
print("Classes:", y.nunique())
```

Output

```
Prepared shape: (614, 11)
Classes: 2
```

The prepared data contain 614 records and 11 features; the label has 2 classes.

3.5 Encoding and Feature Scaling

The six categorical features are converted to 0/1 indicator columns with OneHotEncoder (handle_unknown="ignore"). StandardScaler then transforms each numerical feature to zero mean and unit variance using the training data only. Scaling is applied for Logistic Regression, KNN and SVM, which are sensitive to feature scale; the Decision Tree pipeline has no scaler because tree splits do not depend on feature scale. The indicator columns are not scaled.

3.6 Stratified Training and Testing Split

The data are split into 80% training and 20% testing records with train_test_split. Stratification on the label keeps the approval proportions almost identical in both parts, and random_state=42 makes the split reproducible.

Code

```
# Cell 6: Train/test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42,
stratify=y ) print("Training records:",
len(X_train)) print("Testing records:", len(X_test))
print("Test class counts:", y_test.value_counts().to_dict())
```

Output

```
Training records: 491
Testing records: 123
Test class counts: {'Y': 85, 'N': 38}
```

The training set has 337 approved and 154 not-approved applications (68.6% approved); the testing set has 85 approved and 38 not-approved applications (69.1% approved).

3.7 Final Dataset Summary

Table 3.1: Summary of the prepared data

Item	Value
Records after cleaning	614 (no duplicates or missing labels removed)
Input features	11 (5 numerical: ApplicantIncome, CoapplicantIncome, LoanAmount, Loan_Amount_Term, Credit_History; 6 categorical: Gender, Married, Dependents, Education, Self_Employed, Property_Area)
Target	Loan_Status – 2 classes (Y: 422, N: 192)
Training set	491 records (stratified; 337 Y, 154 N)
Testing set	123 records (stratified; 85 Y, 38 N)
Missing feature values	149 cells, imputed inside the pipelines
Split parameters	test_size = 0.20, random_state = 42, stratify = y

4. Modelling

4.1 Model Selection

The target is a categorical label with two values, so classification models are required. Four classical classifiers from different families were compared on the same data and with the same split: a linear model (Logistic Regression), a rule-based model (Decision Tree), an instance-based model (KNN) and a margin-based model (SVM). No other algorithm was implemented in the notebook.

4.2 Logistic Regression

Logistic Regression estimates the probability of the approved class from a linear combination of the features through the logistic (sigmoid) function. The pipeline consists of the preprocessing step (median / most-frequent imputation, one-hot encoding, StandardScaler) and LogisticRegression(max_iter=2000); the larger iteration limit allows the optimiser to converge.

4.3 Decision Tree

A classification tree repeatedly splits the data with threshold rules on single features and assigns to each final region (leaf) the majority class of the training samples in it. The pipeline uses the same imputation and encoding but no scaler, followed by DecisionTreeClassifier(random_state=42). No depth limit was set, so the tree uses the default settings of scikit-learn.

4.4 K-Nearest Neighbours (KNN)

KNN classifies a new record by the majority class among its k closest training records in feature space. Here k = 5 (KNeighborsClassifier(n_neighbors=5)). Because distances are computed on the numbers themselves, the numerical features are standardised first.

4.5 Support Vector Machine (SVM)

An SVM finds a decision boundary that separates the classes with a large margin. The notebook uses SVC() with default parameters (which in scikit-learn means a radial-basis-function kernel). Features are standardised first. The hyperparameters were not tuned.

4.6 Training Procedure

Each pipeline is fitted on the training set (491 records). For each model the notebook then predicts the 123 test records, stores the fitted pipeline and the predictions, and computes accuracy and macro-averaged precision, recall and F1-score. The results are collected in a table sorted by macro F1-score in descending order.

Code

```
# Cell 7: Train and compare models
def make_preprocessor(scale):
    num_steps = [("imputer", SimpleImputer(strategy="median"))]
if scale:
        num_steps.append(("scaler", StandardScaler()))
return ColumnTransformer([         ("num",
Pipeline(num_steps), numeric_features),
        ("cat", Pipeline([
            ("imputer", SimpleImputer(strategy="most_frequent")),
```

```
            ("onehot", OneHotEncoder(handle_unknown="ignore"))
        ]), categorical_features),
])

models = {     "Logistic
Regression": Pipeline([
        ("prep", make_preprocessor(scale=True)),
        ("model", LogisticRegression(max_iter=2000))
    ]),
    "Decision Tree": Pipeline([
        ("prep", make_preprocessor(scale=False)),
        ("model", DecisionTreeClassifier(random_state=42))
    ]),
    "KNN": Pipeline([
        ("prep", make_preprocessor(scale=True)),
        ("model", KNeighborsClassifier(n_neighbors=5))
    ]),
    "SVM": Pipeline([
        ("prep", make_preprocessor(scale=True)),
        ("model", SVC())
    ])
}

results = [] trained_models = {}
test_predictions = {} for name,
model in models.items():
print(f"Training {name}...")
model.fit(X_train, y_train)
pred = model.predict(X_test)
trained_models[name] = model
test_predictions[name] = pred
results.append({         "Model":
name,
        "Accuracy": accuracy_score(y_test, pred),
        "Precision_macro": precision_score(y_test, pred, average="macro", zero_division=0),
        "Recall_macro": recall_score(y_test, pred, average="macro", zero_division=0),
        "F1_macro": f1_score(y_test, pred, average="macro", zero_division=0)
})

results_df = pd.DataFrame(results).sort_values("F1_macro", ascending=False).reset_index(drop=True)
print(results_df.to_string())
```

Output

```
Training Logistic Regression...
Training Decision Tree...
Training KNN...
Training SVM...
                 Model  Accuracy  Precision_macro  Recall_macro  F1_macro
0                  KNN  0.869919         0.903409      0.796749  0.827489
1                  Logistic Regression  0.861789         0.898261      0.783591  0.814710
2                  SVM  0.853659         0.893114      0.770433  0.801613
3                  Decision Tree  0.756098         0.714396      0.714396  0.714396
```

4.7 Model Comparison

Table 4.1: Configuration of the four models

Model	**Pipeline steps**	**Settings**
Logistic Regression	Imputer + one-hot encoder + StandardScaler → LogisticRegression	max_iter = 2000
Decision Tree	Imputer + one-hot encoder → DecisionTreeClassifier	random_state = 42 (no depth limit)
KNN	Imputer + one-hot encoder + StandardScaler → KNeighborsClassifier	n_neighbors = 5
SVM	Imputer + one-hot encoder + StandardScaler → SVC	Defaults (RBF kernel)

5. Evaluation

5.1 Evaluation Strategy

All models are evaluated on the same stratified test set of 123 records that were not used for training. Since this is a classification problem, classification metrics are used. The models are ranked by macro F1-score, and the first row of the sorted table is selected as the best model. The notebook does not use cross-validation or a separate validation set. For a class c, let TP, FP and FN denote the numbers of true positives, false positives and false negatives. Macro averages give each of the two classes the same weight, which is useful here because the not-approved class is the minority.

5.2 Metrics

Accuracy is the fraction of test records predicted correctly. For each class, precision = TP / (TP + FP) and recall = TP / (TP + FN), and F1 = 2 × precision × recall / (precision + recall). Macro averages give the two classes equal weight, and macro F1 was the model-selection criterion.

5.6 Evaluation Table

Table 5.1: Test-set evaluation results (notebook output)

Rank	Model	Accuracy	Precision (macro)	Recall (macro)	F1 (macro)
1	KNN	0.869919	0.903409	0.796749	0.827489
2	Logistic Regression	0.861789	0.898261	0.783591	0.814710
3	SVM	0.853659	0.893114	0.770433	0.801613
4	Decision Tree	0.756098	0.714396	0.714396	0.714396

Code

```
# Cell 8: Plot model comparison results_df.set_index("Model")[["Accuracy", "Precision_macro",
"Recall_macro", "F1_macro"]].plot(     kind="bar", figsize=(12, 5) ) plt.title("Loan Approval
Model Comparison") plt.ylabel("Score") plt.ylim(0, 1.05) plt.xticks(rotation=20)
plt.tight_layout() plt.show()
```

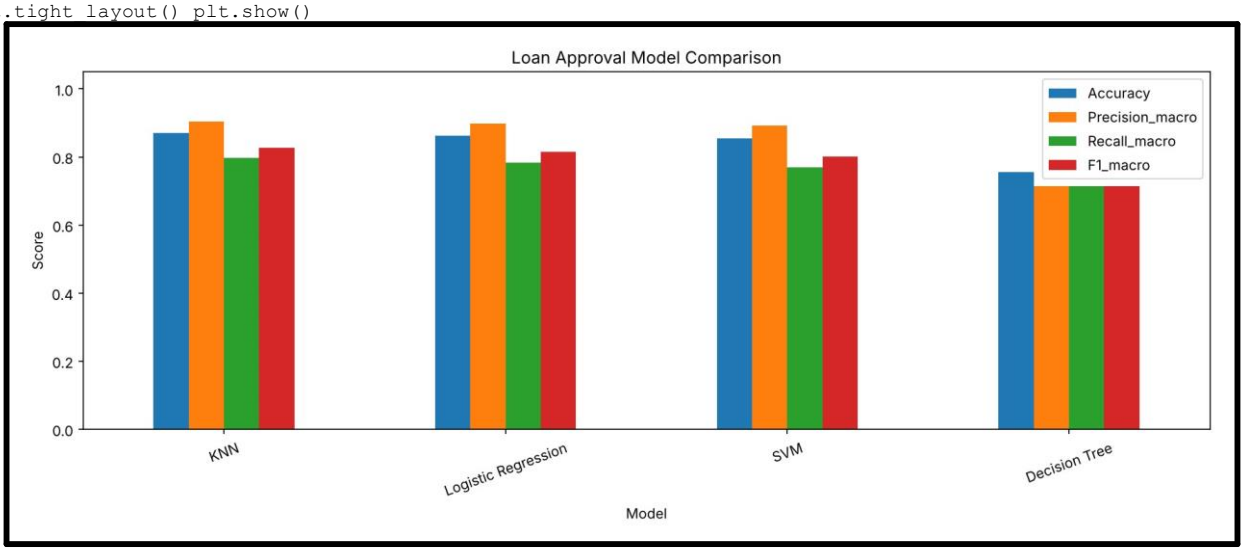


Figure 5.1: Loan approval model comparison (notebook output)

5.7 Interpretation of Results

- **Selected model:** KNN obtained the highest macro F1-score (0.827489) and also the highest accuracy (0.869919), macro precision (0.903409) and macro recall (0.796749), and was selected.
- **Small differences among the top three:** with 123 test records, the accuracy of 0.869919 corresponds to 107 correct predictions, against 106 for Logistic Regression, 105 for SVM and 93 for the Decision Tree (counts derived from the accuracy values). KNN, Logistic Regression and SVM are therefore separated by only 1 to 2 records, and the ranking among them is not conclusive.
- **Decision Tree is clearly weakest:** it is 14 records behind KNN. A tree grown without a depth limit can fit the training data too closely, which is consistent with this result, although the notebook does not test it.

- **Comparison with a trivial baseline:** answering Y for every application would be correct for 85 of 123 test records (accuracy 0.6911). KNN, Logistic Regression and SVM beat this by about 16 percentage points; the Decision Tree by only about 6.
- **Macro recall is lower than accuracy:** because the test classes are unequal (85 Y and 38 N), macro recall averages the two class recalls (0.6053 for N and 0.9882 for Y for KNN) and is 0.7967, below the accuracy of 0.8699.
- **Overall:** the three best models are fairly accurate (about 85–87%), but all of them find approvals much more reliably than rejections (Section 5.8). The ranking reflects small differences on a single split of a modest test set, so it does not prove that KNN would always be best.

5.8 Classification Report of the Selected Model

Code

```
# Cell 9: Evaluate the selected model
best_name = results_df.iloc[0]["Model"]
best_model = trained_models[best_name]
best_pred = test_predictions[best_name]
print("Selected model:", best_name)
print("\nClassification report:")
print(classification_report(y_test, best_pred, zero_division=0))
```

Output

```
Selected model: KNN

Classification report:               precision
recall  f1-score   support

           N       0.96      0.61      0.74        38
Y       0.85      0.99      0.91        85

    accuracy                           0.87       123
macro avg       0.90      0.80      0.83       123
weighted avg       0.88      0.87      0.86       123
```

Table 5.2: Classification report for KNN (notebook output)

Class / average	Precision	Recall	F1-score	Support
N (not approved)	0.96	0.61	0.74	38
Y (approved)	0.85	0.99	0.91	85
accuracy			0.87	123
macro avg	0.90	0.80	0.83	123
weighted avg	0.88	0.87	0.86	123

Approved applications are recognised very well (recall 0.99), but only 61% of the not-approved applications are found (recall 0.61). When the model says N it is almost always right (precision 0.96), yet it says Y for many applications that were in fact rejected (precision for Y is 0.85).

5.9 Confusion Matrix

Code

```
# Cell 10: Confusion matrix labels =
sorted(y.unique()) cm = confusion_matrix(y_test,
best_pred, labels=labels) plt.figure(figsize=(5.5,
4.5))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
xticklabels=labels, yticklabels=labels)
plt.title(f"Confusion Matrix - {best_name}")
plt.xlabel("Predicted status") plt.ylabel("Actual
status") plt.tight_layout() plt.show()
```

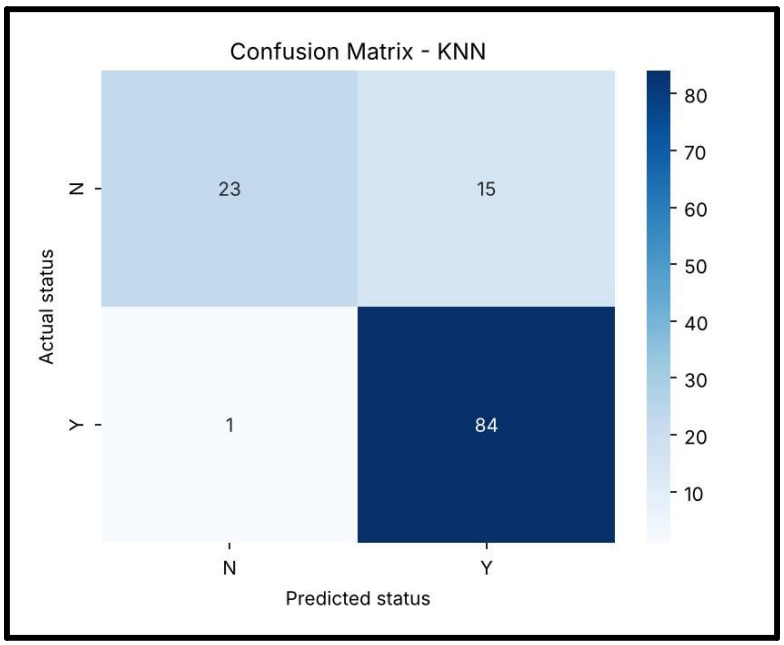


Figure 5.2: Confusion matrix – KNN (notebook output)

16 of the 123 test records are misclassified, and the confusion matrix shows exactly where: 15 not-approved applications were predicted as approved (false approvals) and 1 approved application was predicted as not approved. The other 107 records (23 correctly rejected and 84 correctly approved) lie on the diagonal. For a lender the 15 false approvals are the costly kind of error, and they make up 15 of the 16 mistakes.

5.10 Limitations

- The test set is small (123 records, only 38 not approved), so a single misclassified record changes the recall of the N class by about 0.026.
- Only one stratified split was used and no cross-validation was run, so the stability of the results is not assessed.
- The best model was chosen using the test set and the same test set was used to report its performance, so the reported performance of the selected model may be slightly optimistic. A separate validation set or cross-validation would avoid this.
- No hyperparameters were tuned (KNN uses k = 5, the SVM and the Decision Tree use defaults), and no class weighting or threshold adjustment was tried to improve the recall of the not-approved class.
- Gender and Married are used as input features because they are part of the dataset. Using such personal attributes in a real lending decision raises fairness and legal concerns, and the model was not audited for bias.
- The data come from one source of 614 applications with many missing values filled by simple rules; real lending data are larger and noisier, and the results should not be assumed to carry over.

6. Deployment and Final Project Summary

6.1 Saving the Selected Model

The selected pipeline (KNN with its imputers, encoder and scaler) is saved together with the feature names, the list of classes and the model name in one dictionary.

Code

```
# Cell 11: Save the
model model_package = {
"model": best_model,
    "features": required_features,
    "classes": labels,
    "best_model_name": best_name }
joblib.dump(model_package,
"loan_approval_model.pkl") print("Saved
loan_approval_model.pkl")
```

Output

```
Saved loan_approval_model.pkl
```

6.2 Model File Format

The file loan_approval_model.pkl is a joblib-serialised (pickle-based) Python dictionary with four keys: model (the fitted pipeline), features (the ordered list of eleven feature names), classes (the sorted list ['N', 'Y']) and best_model_name. It can be restored with joblib.load in an environment with a compatible scikit-learn version and should only be loaded from trusted sources. The notebook also reloads the file and checks that the reloaded model gives the same predictions as the in-memory model (Section 6.5).

6.3 Loan Prediction Function

The function predict_loan takes the eleven application details, assembles a one-row DataFrame with the columns in the training order, and returns the predicted status. If the model offers probability estimates, it also prints the confidence of the prediction. The selected KNN pipeline does provide probability estimates; for KNN these are the share of the five nearest neighbours that belong to the predicted class, so the values can only be 60%, 80% or 100%.

Code

```
# Cell 12: Loan approval prediction function
def predict_loan(ApplicantIncome, CoapplicantIncome, LoanAmount,
Loan_Amount_Term,                  Credit_History, Gender, Married, Dependents,
Education,                  Self_Employed, Property_Area):     input_data =
pd.DataFrame([{         "ApplicantIncome": ApplicantIncome,
        "CoapplicantIncome": CoapplicantIncome,
        "LoanAmount": LoanAmount,
        "Loan_Amount_Term": Loan_Amount_Term,
        "Credit_History": Credit_History,
        "Gender": Gender,
        "Married": Married,
        "Dependents": Dependents,
        "Education": Education,
        "Self_Employed": Self_Employed,
        "Property_Area": Property_Area     }], columns=required_features)     prediction =
best_model.predict(input_data)[0]     print("Predicted loan status:", "Approved (Y)" if
prediction == "Y" else "Rejected (N)")     if hasattr(best_model, "predict_proba"):
        probabilities = best_model.predict_proba(input_data)[0]
print(f"Model confidence: {probabilities.max() * 100:.2f}%")     return
prediction

# Example inputs for demonstration predict_loan(
ApplicantIncome=5000, CoapplicantIncome=1500, LoanAmount=130,
    Loan_Amount_Term=360, Credit_History=1, Gender="Male", Married="Yes",
    Dependents="0", Education="Graduate", Self_Employed="No",
    Property_Area="Semiurban" ) predict_loan(
ApplicantIncome=3000, CoapplicantIncome=0, LoanAmount=150,
    Loan_Amount_Term=360, Credit_History=0, Gender="Female", Married="No",
    Dependents="0", Education="Not Graduate", Self_Employed="No",
    Property_Area="Rural"
)
```

Output

```
Predicted loan status: Approved (Y)
Model confidence: 100.00%
Predicted loan status: Rejected (N)
Model confidence: 100.00%
```

6.4 Example Predictions

Table 6.1: Example predictions (notebook output)

Item	Example 1	Example 2
ApplicantIncome / CoapplicantIncome	5000 / 1500	3000 / 0
LoanAmount / Loan_Amount_Term	130 / 360	150 / 360
Credit_History	1	0
Gender / Married / Dependents	Male / Yes / 0	Female / No / 0
Education / Self_Employed	Graduate / No	Not Graduate / No

Property_Area	Semiurban	Rural
Model used	KNN (selected model)	KNN (selected model)
Predicted status	Approved (Y)	Rejected (N)
Model confidence	100.00%	100.00%

These are two invented applications chosen for demonstration; the notebook has no ground-truth label for them, so their correctness cannot be verified. The outcomes agree with the pattern seen in Section 2.6.4 (credit history 1 with a semiurban property versus credit history 0 with a rural property), and the 100% confidence only means that all five nearest neighbours agreed, not that the decision is certain.

6.5 Deployment Limitations

- No website, API or production application was implemented; "deployment" in this project means a saved model file and a notebook function.
- The saved file was re-loaded and reproduced all 123 test predictions of the in-memory model (Cell 13, output: True), but the prediction function itself still uses the notebook variable best_model rather than the loaded file.
- The confidence value is a neighbour vote share, not a calibrated probability.
- The saved file is tied to the scikit-learn version used for training and is a pickle-based file, which must only be loaded from trusted sources.
- The model was trained on one small dataset; its predictions must not replace a lender's credit assessment or legal requirements.

Cell 13 reloads the saved file and compares predictions:

Code

```
# Cell 13: Reload test (verifies the saved file) pkg =
joblib.load("loan_approval_model.pkl") reload_pred =
pkg["model"].predict(X_test) reload_match =
bool((reload_pred == best_pred).all()) print("Reloaded model
reproduces predictions:", reload_match)
```

Output

```
Reloaded model reproduces predictions: True
```

7. Conclusion

7.1 Summary of Results

An end-to-end CRISP-DM pipeline was built to predict loan approval from eleven application attributes. The dataset of 614 records had no duplicates, 149 missing cells spread over seven columns, and 68.7% approved applications. It was split in a stratified way into 491 training and 123 testing records. Logistic Regression, Decision Tree, KNN and SVM were trained with comparable preprocessing and compared. KNN achieved the best results (accuracy 0.869919, macro precision 0.903409, macro recall 0.796749, macro F1 0.827489), followed by Logistic Regression, SVM and the Decision Tree (accuracy 0.756098). The selected model misclassified 16 of 123 test records, 15 of them rejected applications that it predicted as approved. It was saved to a file, reloaded successfully and used in two prediction examples.

7.2 Key Findings

- Three of the four models reach 85–87% accuracy, about 16 percentage points above the 69% obtained by always predicting approval; the Decision Tree reaches only 75.6%.
- The differences among KNN, Logistic Regression and SVM are small (1–2 test records), so their ranking should be interpreted with caution.
- Credit history is the attribute most clearly linked to approval in the exploratory analysis (79.6% approved with a good credit history against 7.9% without).
- The selected model finds 99% of the approved applications but only 61% of the rejected ones, so accuracy overstates how well it handles the minority class.

7.3 Future Enhancements

- Use cross-validation and a separate validation set for model selection, and report the test set only once.

- Tune hyperparameters (for example k for KNN, the regularisation and kernel parameters of the SVM, and the tree depth).
- Address the class imbalance with class weights, re-sampling or an adjusted decision threshold to raise the recall of the not-approved class.
- Evaluate additional classifiers such as Random Forest or gradient boosting, and compare them with the four classical models.
- Engineer features such as total income or the loan-to-income ratio, and review whether personal attributes such as gender should be used at all.
- Package the pipeline as an application that loads the saved model from disk.

8. Appendix

8.1 Python Libraries

Code

```
# Cell 1: Import libraries import warnings import numpy as np
import pandas as pd import matplotlib.pyplot as plt import
seaborn as sns import joblib from sklearn.model_selection
import train_test_split from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer from
sklearn.impute import SimpleImputer from sklearn.preprocessing
import StandardScaler, OneHotEncoder from sklearn.linear_model
import LogisticRegression from sklearn.tree import
DecisionTreeClassifier from sklearn.neighbors import
KNeighborsClassifier from sklearn.svm import SVC from
sklearn.metrics import (     accuracy_score, precision_score,
recall_score, f1_score,     classification_report,
confusion_matrix ) warnings.filterwarnings("ignore")
print("Libraries imported successfully.")
```

Output

```
Libraries imported successfully.
```

Table 8.1: Libraries used in the notebook

Library / module	Purpose
pandas	Loading, cleaning and preparing the dataset
numpy	Numerical operations (selecting numeric columns for the correlation matrix)
matplotlib	Plots (status distribution, missing values, approval rates, model comparison, confusion matrix)
seaborn	Heatmaps (correlation matrix, confusion matrix)
scikit-learn	train_test_split, Pipeline, ColumnTransformer, SimpleImputer, StandardScaler, OneHotEncoder, LogisticRegression, DecisionTreeClassifier, KNeighborsClassifier, SVC, and the classification metrics
joblib	Saving and reloading the selected model

8.2 Important Code Snippets

The key code of each step is shown in the section where it is used (Cells 1–13). Lines that save figures or record results for this report were replaced by plt.show().

8.3 Model Evaluation Results

Table 8.5: Differences from the selected model (KNN)

Model	Accuracy	Macro F1	Accuracy gap to KNN	Correct of 123 (derived)	Recall of N	False approvals
KNN	0.869919	0.827489	–	107	0.61	15
Logistic Regression	0.861789	0.814710	0.008130	106	0.58	16
SVM	0.853659	0.801613	0.016260	105	0.55	17

Decision Tree	0.756098	0.714396	0.113821	93	0.61	15

"False approvals" are not-approved test applications that the model predicted as approved.

8.4 Example Prediction

Input: ApplicantIncome = 5000, CoapplicantIncome = 1500, LoanAmount = 130, Loan_Amount_Term = 360, Credit_History = 1, Gender = Male, Married = Yes, Dependents = 0, Education = Graduate, Self_Employed = No, Property_Area = Semiurban. Output: Approved (Y), predicted by the KNN pipeline (see Section 6.4 for the interpretation).

References

1. Analytics Vidhya, Loan Prediction practice problem – training dataset (train_u6lujuX_CVtuZ9i.csv). Also circulated on Kaggle.
2. Chapman, P., Clinton, J., Kerber, R., Khabaza, T., Reinartz, T., Shearer, C. and Wirth, R. (2000). CRISP-DM 1.0: Step-by-step data mining guide. SPSS Inc.
3. Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825–2830.